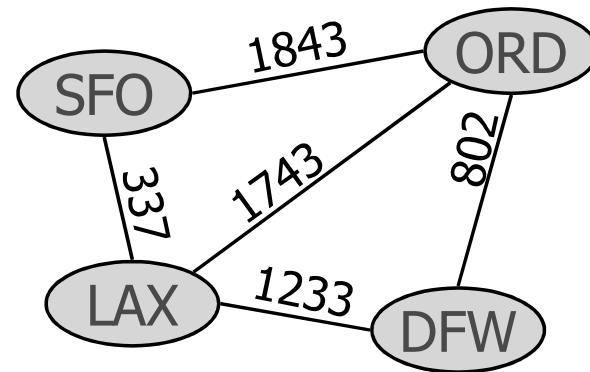


Graphs

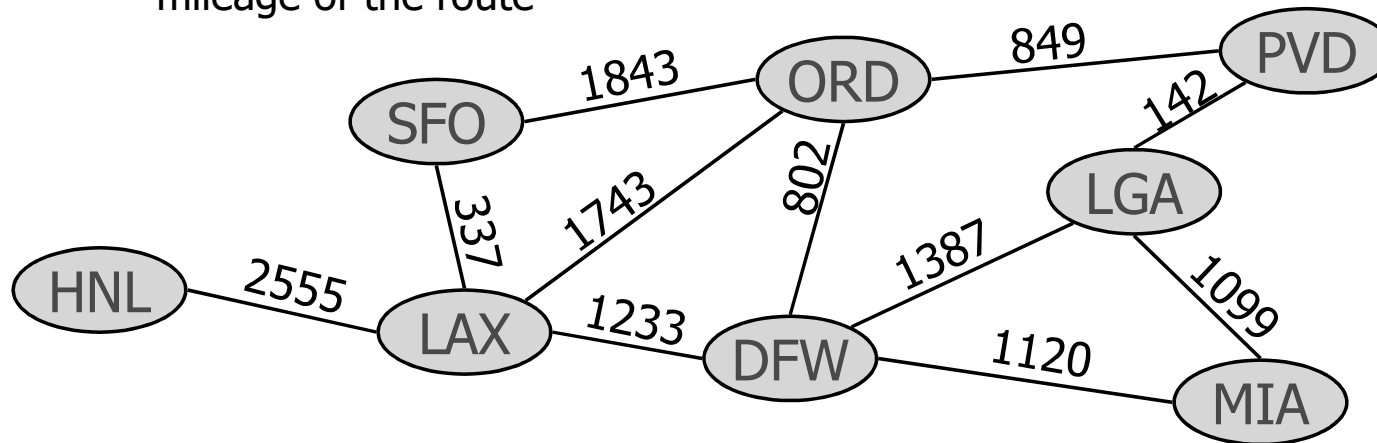


Graphs

- ◆ Graphs are collections of nodes in which various pairs of nodes are connected by line segments.
- ◆ The nodes are usually called vertices and
- ◆ the line segments are called edges

Graphs (§ 12.1)

- ◆ A graph is a pair (V, E) , where
 - V is a set of nodes, called vertices
 - E is a collection of pairs of vertices, called edges
 - Vertices and edges are positions and store elements
- ◆ Example:
 - A vertex represents an airport and stores the three-letter airport code
 - An edge represents a flight route between two airports and stores the mileage of the route



Directed Graph and Undirected Graph

◆ A directed graph or digraph

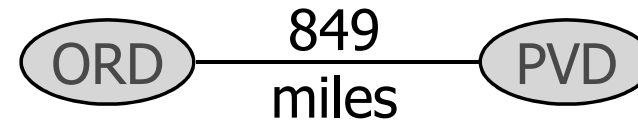
- is a graph in which each edge has a direction(arrow head) to its successor.

◆ Undirected graph

- Is a graph in which there is no direction (arrow head) on any edge
- The flow between two vertices can go in either direction

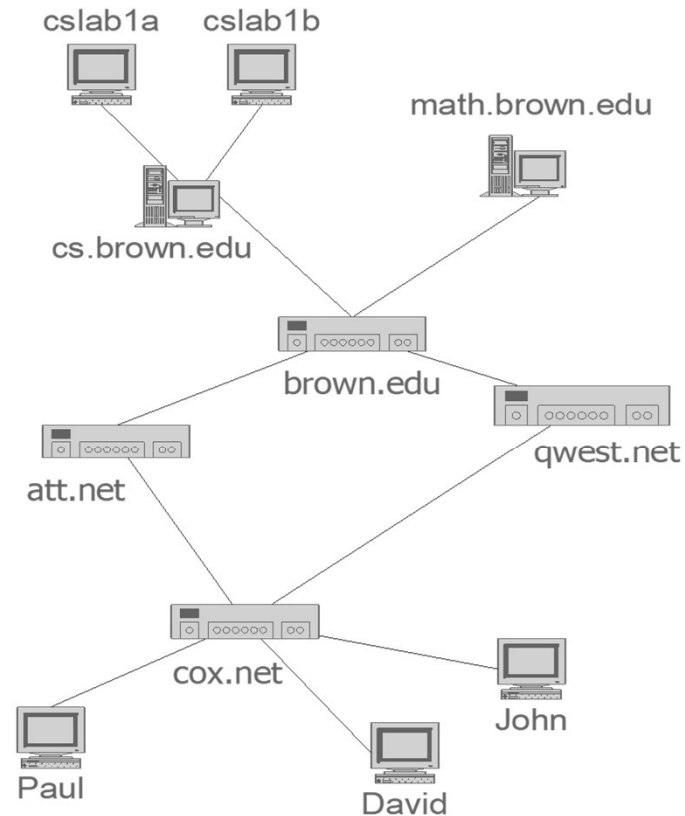
Edge Types

- ◆ Directed edge
 - ordered pair of vertices (u,v)
 - first vertex u is the origin
 - second vertex v is the destination
 - e.g., a flight
- ◆ Undirected edge
 - unordered pair of vertices (u,v)
 - e.g., a flight route
- ◆ Directed graph
 - all the edges are directed
 - e.g., route network
- ◆ Undirected graph
 - all the edges are undirected
 - e.g., flight network



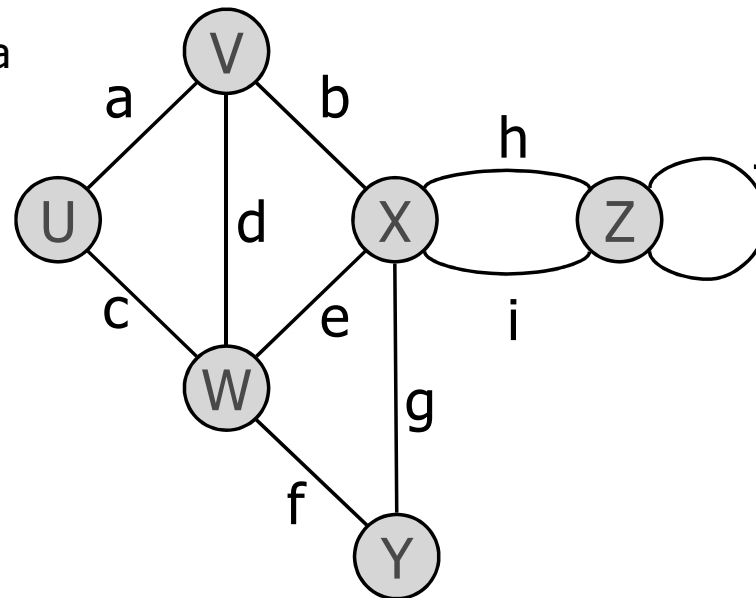
Applications

- ◆ Electronic circuits
 - Printed circuit board
 - Integrated circuit
- ◆ Transportation networks
 - Highway network
 - Flight network
- ◆ Computer networks
 - Local area network
 - Internet
 - Web
- ◆ Databases
 - Entity-relationship diagram



Terminology

- ◆ End vertices (or endpoints) of an edge
 - U and V are the endpoints of a
- ◆ Edges incident on a vertex
 - a, d, and b are incident on V
- ◆ Adjacent vertices
 - U and V are adjacent
- ◆ Degree of a vertex
 - X has degree 5
- ◆ Parallel edges
 - h and i are parallel edges
- ◆ Self-loop
 - j is a self-loop



Terminology (cont.)

◆ Path

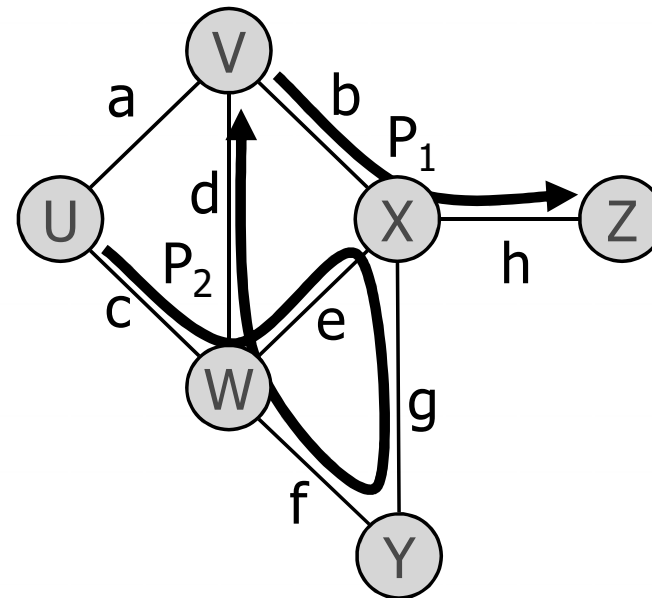
- sequence of alternating vertices and edges
- begins with a vertex
- ends with a vertex
- each edge is preceded and followed by its endpoints

◆ Simple path

- path such that all its vertices and edges are distinct

◆ Examples

- $P_1 = (V, b, X, h, Z)$ is a simple path
- $P_2 = (U, c, W, e, X, g, Y, f, W, d, V)$ is a path that is not simple



Terminology (cont.)

◆ Cycle

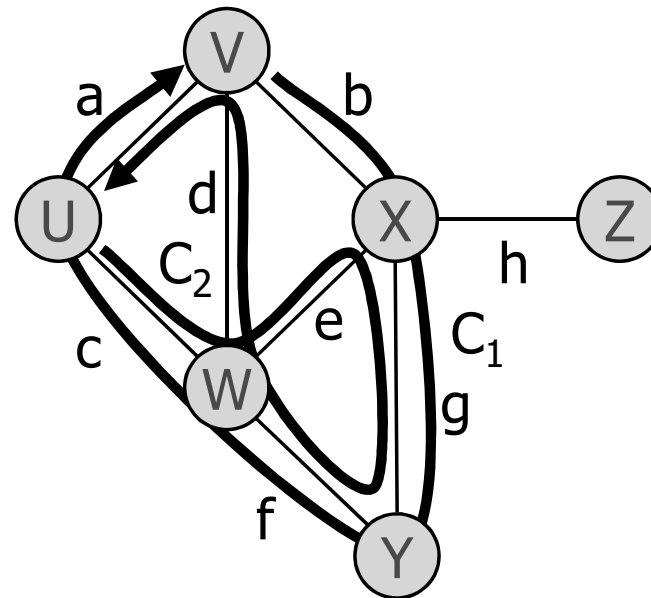
- circular sequence of alternating vertices and edges
- each edge is preceded and followed by its endpoints

◆ Simple cycle

- cycle such that all its vertices and edges are distinct

◆ Examples

- $C_1 = (V, b, X, g, Y, f, W, c, U, a, \downarrow)$ is a simple cycle
- $C_2 = (U, c, W, e, X, g, Y, f, W, d, V, a, \downarrow)$ is a cycle that is not simple



Properties

Property 1

$$\sum_v \deg(v) = 2m$$

Proof: each edge is counted twice

Property 2

In an undirected graph with no self-loops and no multiple edges

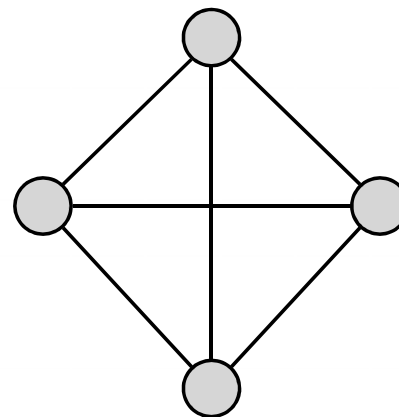
$$m \leq n(n-1)/2$$

Proof: each vertex has degree at most $(n-1)$

What is the bound for a directed graph?

Notation

n	number of vertices
m	number of edges
$\deg(v)$	degree of vertex v



Example

- $n = 4$
- $m = 6$
- $\deg(v) = 3$

Operations

1. Add vertex

- a) inserts a new vertex into the graph

2. Delete vertex

- a) Removes a vertex from the graph
- b) All connection edges are also removed

3. Add Edge

- a) Connects a vertex to a destination vertex.
- b) If vertex requires multiple edges, then add an edge must be called once for each adjacent vertex.
- c) To add edge, 2 vertices must be specified.
- d) If the graph is DIGRAPH, then one of the vertices must be specified as source and one as destination

4. Delete edge

Delete an edge from a graph

5. Find vertex

Traverses a graph looking for a specified vertex.

If the vertex is found, its address is return else return error

Main Methods of the Graph ADT

◆ Vertices and edges

- are positions
- store elements

◆ Accessor methods

- `endVertices(e)`: an array of the two endvertices of `e`
- `opposite(v, e)`: the vertex opposite of `v` on `e`
- `areAdjacent(v, w)`: true iff `v` and `w` are adjacent
- `replace(v, x)`: replace element at vertex `v` with `x`
- `replace(e, x)`: replace element at edge `e` with `x`

◆ Update methods

- `insertVertex(o)`: insert a vertex storing element `o`
- `insertEdge(v, w, o)`: insert an edge `(v,w)` storing element `o`
- `removeVertex(v)`: remove vertex `v` (and its incident edges)
- `removeEdge(e)`: remove edge `e`

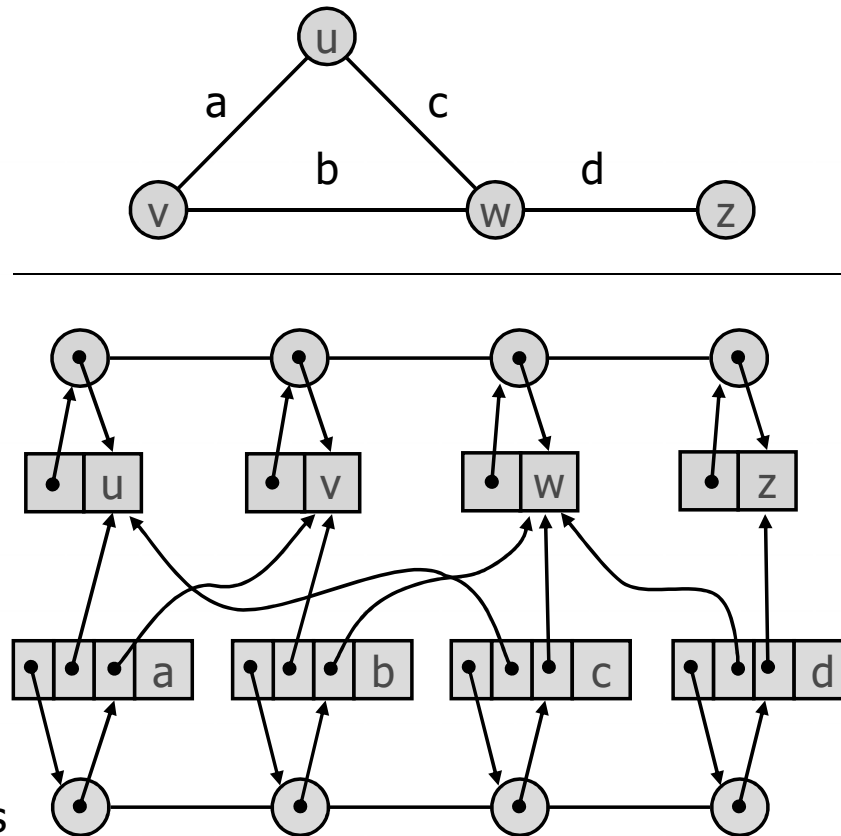
◆ Iterator methods

- `incidentEdges(v)`: edges incident to `v`
- `vertices()`: all vertices in the graph
- `edges()`: all edges in the graph

Edge List Structure

(§ 12.2.1)

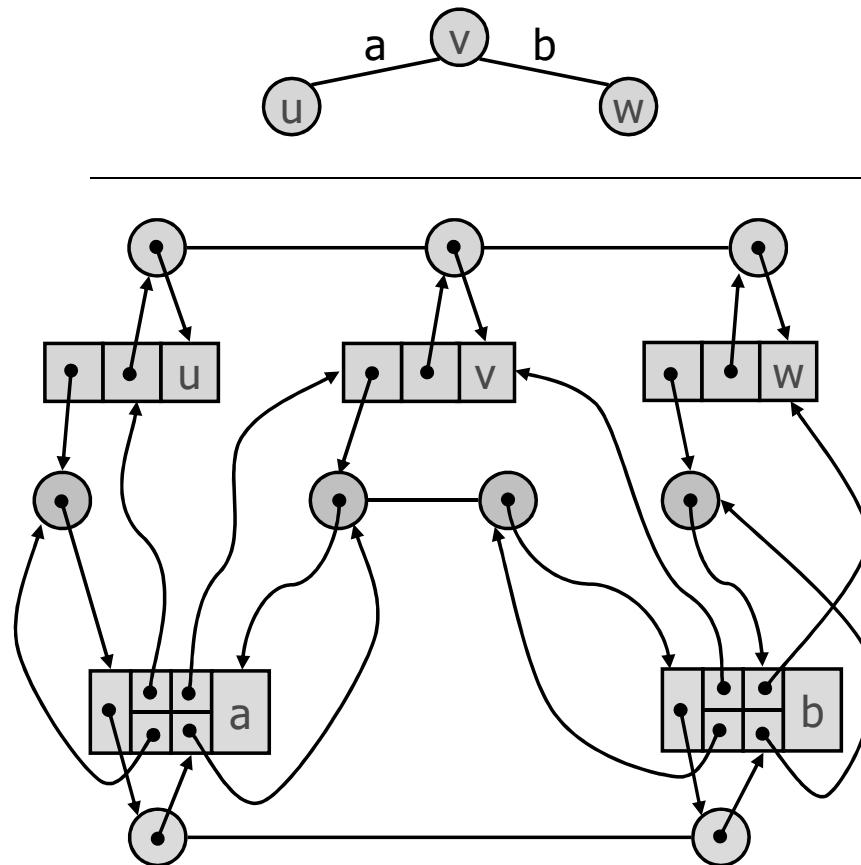
- ◆ Vertex object
 - element
 - reference to position in vertex sequence
- ◆ Edge object
 - element
 - origin vertex object
 - destination vertex object
 - reference to position in edge sequence
- ◆ Vertex sequence
 - sequence of vertex objects
- ◆ Edge sequence
 - sequence of edge objects



Adjacency List Structure

(§ 12.2.2)

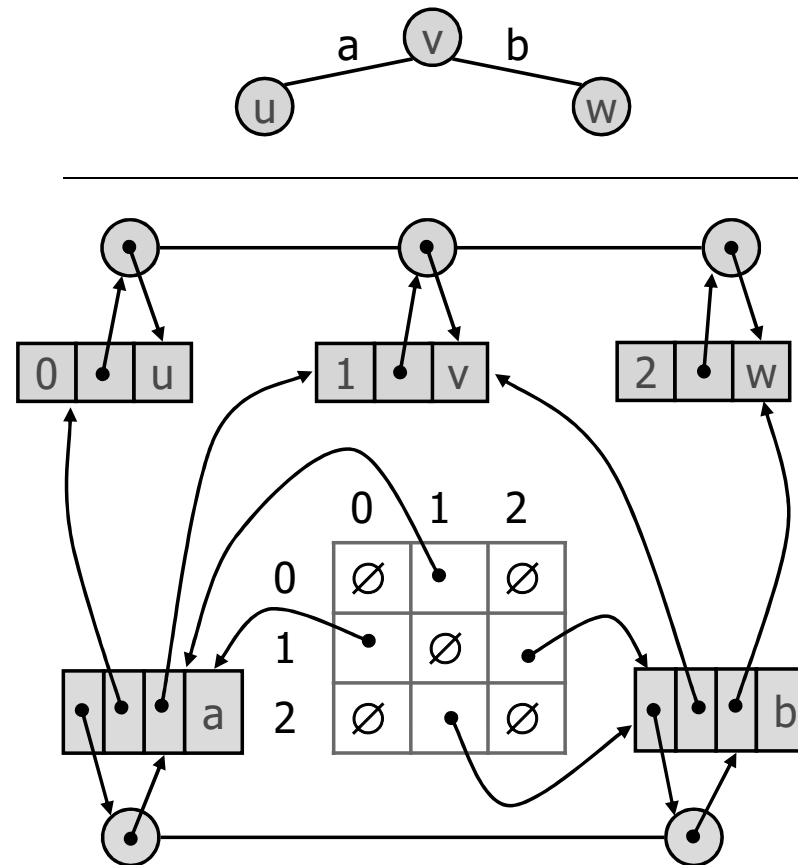
- ◆ Edge list structure
- ◆ Incidence sequence for each vertex
 - sequence of references to edge objects of incident edges
- ◆ Augmented edge objects
 - references to associated positions in incidence sequences of end vertices



Adjacency Matrix Structure

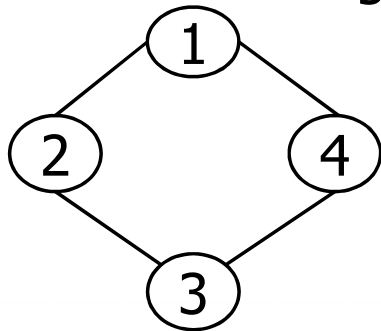
(§ 12.2.3)

- ◆ Edge list structure
- ◆ Augmented vertex objects
 - Integer key (index) associated with vertex
- ◆ 2D-array adjacency array
 - Reference to edge object for adjacent vertices
 - Null for non adjacent vertices
- ◆ The “old fashioned” version just has 0 for no edge and 1 for edge

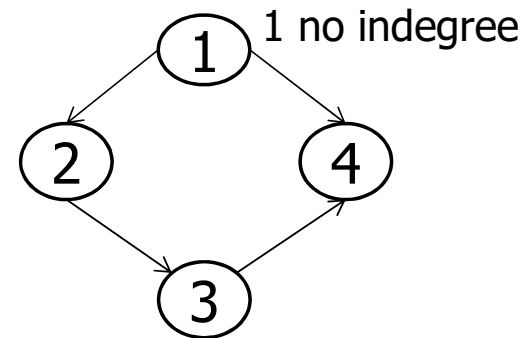


Adjacency Matrix

◆ Is 2-Dimensional Array which has the size $V \times V$ (V are the number of vertices in the graph)



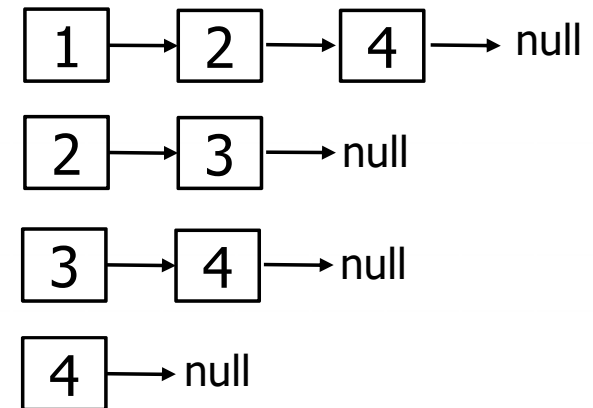
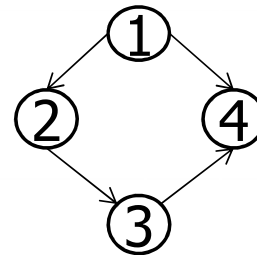
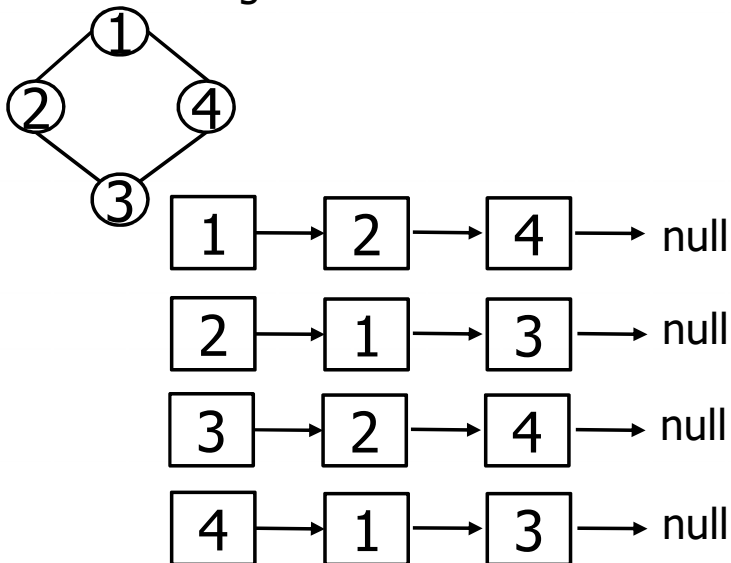
	1	2	3	4
1	0	1	0	1
2	1	0	1	0
3	0	1	0	1
4	1	0	1	0



	1	2	3	4
1	0	1	0	1
2	0	0	2	0
3	0	0	0	3
4	0	0	0	0

Adjacency List

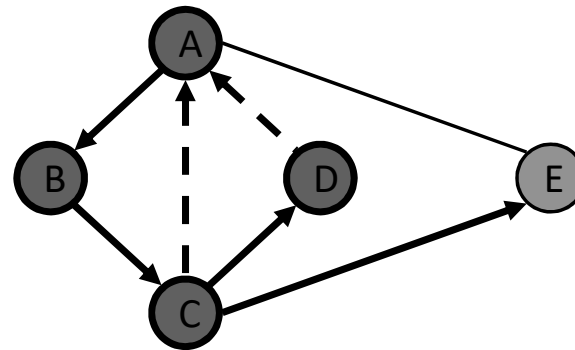
- ◆ Adjacency List is the Array[] of Linked List, where array size is same as number of Vertices in the graph.
- ◆ Every Vertex has a Linked List. Each Node in this Linked list represents the reference to the other vertices which share an edge with the current vertex.
- ◆ The weights can also be stored in the Linked List Node.



Graph Traversals

- ◆ Depth-First Search
- ◆ Breadth-First Search

Depth-First Search



Depth-First Search (§ 12.3.1)

- Depth-first search (DFS) is a general technique for traversing a graph
- A DFS traversal of a graph G
 - Visits all the vertices and edges of G
 - Determines whether G is connected
 - Computes the connected components of G
 - Computes a spanning forest of G
- DFS on a graph with n vertices and m edges takes $O(n + m)$ time
- DFS can be further extended to solve other graph problems
 - Find and report a path between two given vertices
 - Find a cycle in the graph
- Depth-first search is to graphs what Euler tour is to binary trees

DFS Algorithm

- The algorithm uses a mechanism for setting and getting “labels” of vertices and edges

Algorithm *DFS*(*G*)

Input graph *G*

Output labeling of the edges of *G*
as discovery edges and
back edges

```
for all u ∈ G.vertices()  
    setLabel(u, UNEXPLORED)  
for all e ∈ G.edges()  
    setLabel(e, UNEXPLORED)  
for all v ∈ G.vertices()  
    if getLabel(v) = UNEXPLORED  
        DFS(G, v)
```

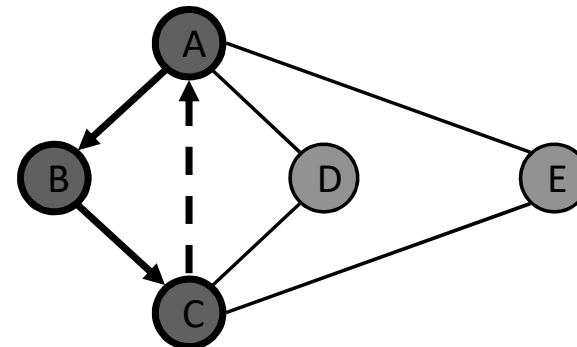
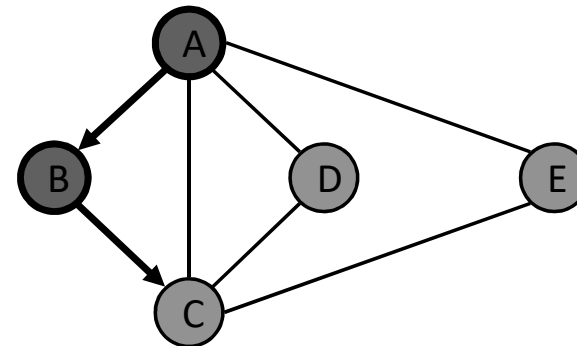
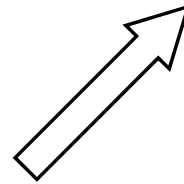
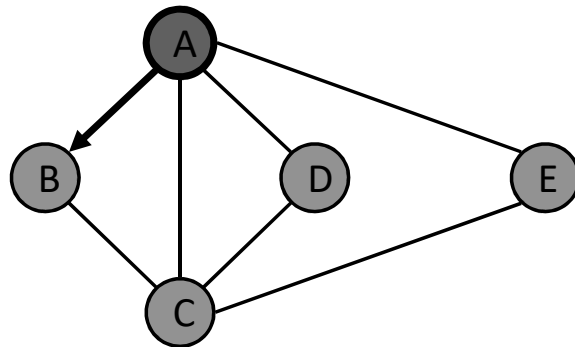
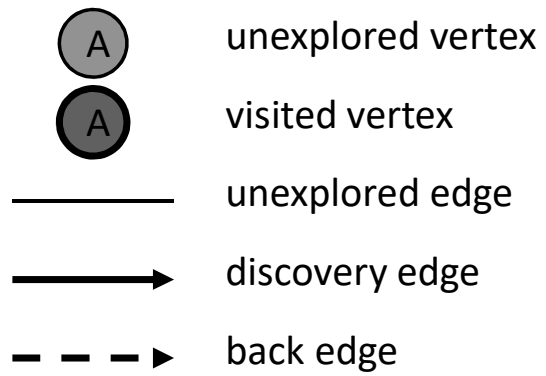
Algorithm *DFS*(*G*, *v*)

Input graph *G* and a start vertex *v* of *G*

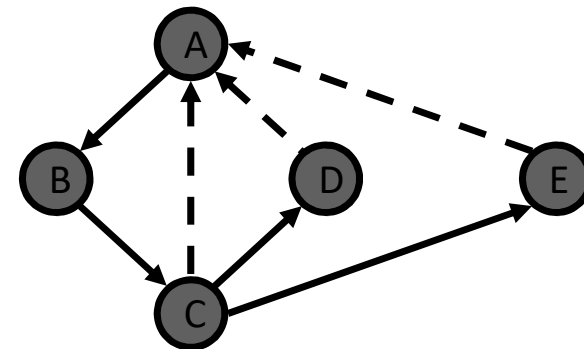
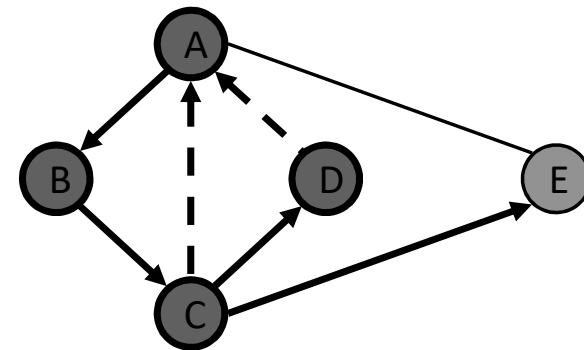
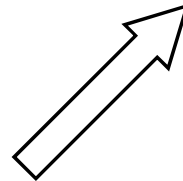
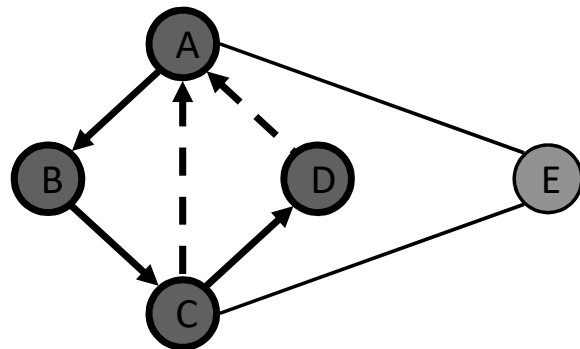
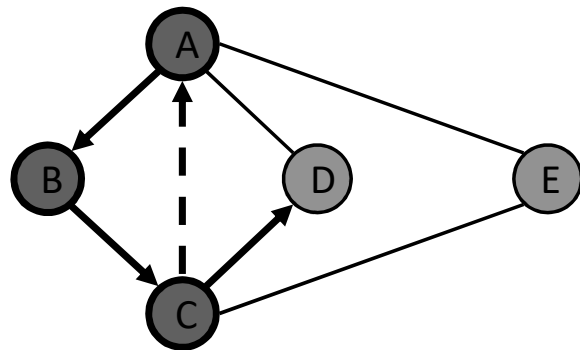
Output labeling of the edges of *G*
in the connected component of *v*
as discovery edges and back edges

```
setLabel(v, VISITED)  
for all e ∈ G.incidentEdges(v)  
    if getLabel(e) = UNEXPLORED  
        w ← opposite(v, e)  
        if getLabel(w) = UNEXPLORED  
            setLabel(e, DISCOVERY)  
            DFS(G, w)  
        else  
            setLabel(e, BACK)
```

Example



Example (cont.)



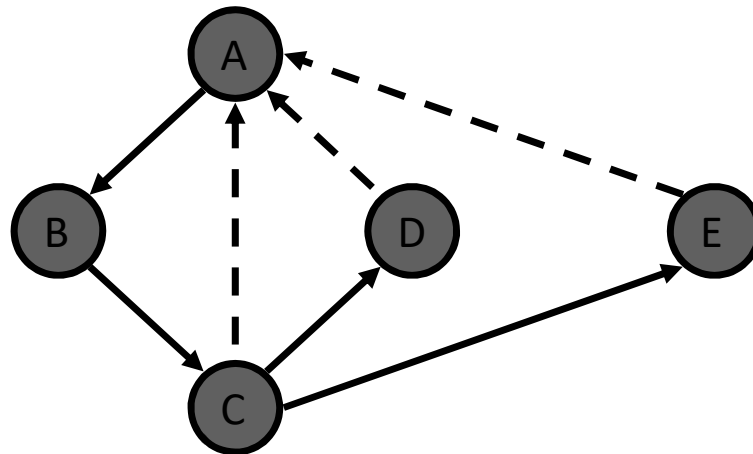
Properties of DFS

Property 1

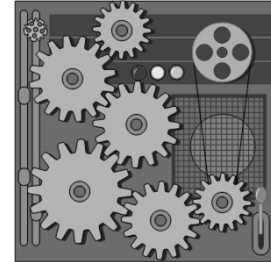
$DFS(G, v)$ visits all the vertices and edges in the connected component of v

Property 2

The discovery edges labeled by $DFS(G, v)$ form a spanning tree of the connected component of v



Analysis of DFS



- Setting/getting a vertex/edge label takes $O(1)$ time
- Each vertex is labeled twice
 - once as UNEXPLORED
 - once as VISITED
- Each edge is labeled twice
 - once as UNEXPLORED
 - once as DISCOVERY or BACK
- Method incidentEdges is called once for each vertex
- DFS runs in $O(n + m)$ time provided the graph is represented by the adjacency list structure
 - Recall that $\sum_v \deg(v) = 2m$

Path Finding



- We can specialize the DFS algorithm to find a path between two given vertices u and z using the template method pattern
- We call $DFS(G, u)$ with u as the start vertex
- We use a stack S to keep track of the path between the start vertex and the current vertex
- As soon as destination vertex z is encountered, we return the path as the contents of the stack

```
Algorithm pathDFS( $G, v, z$ )  
  setLabel( $v, VISITED$ )  
   $S.push(v)$   
  if  $v = z$   
    return  $S.elements()$   
  for all  $e \in G.incidentEdges(v)$   
    if getLabel( $e$ ) = UNEXPLORED  
       $w \leftarrow opposite(v, e)$   
      if getLabel( $w$ ) = UNEXPLORED  
        setLabel( $e, DISCOVERY$ )  
         $S.push(e)$   
        pathDFS( $G, w, z$ )  
         $S.pop(e)$   
      else  
        setLabel( $e, BACK$ )  
   $S.pop(v)$ 
```

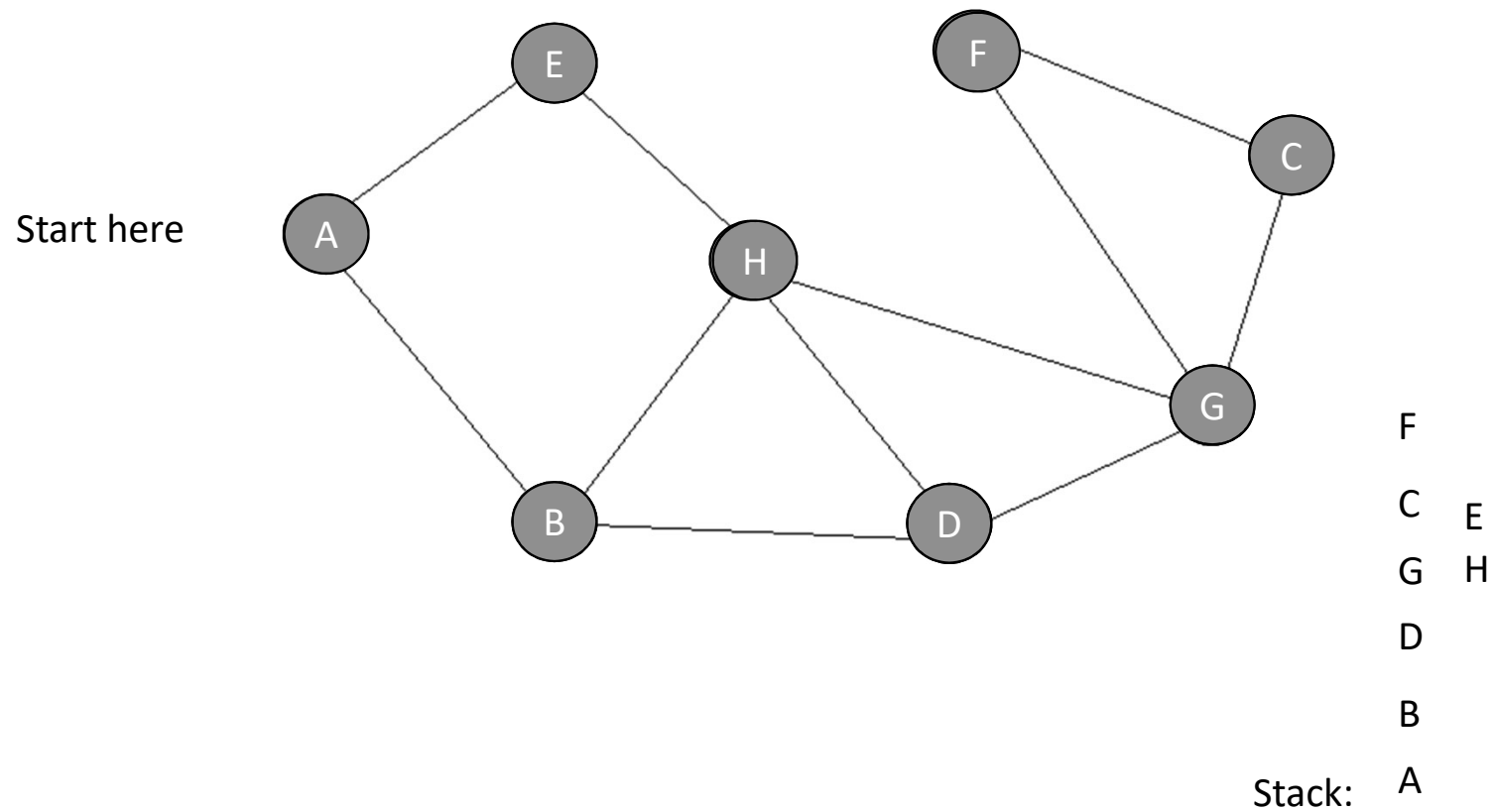
Cycle Finding



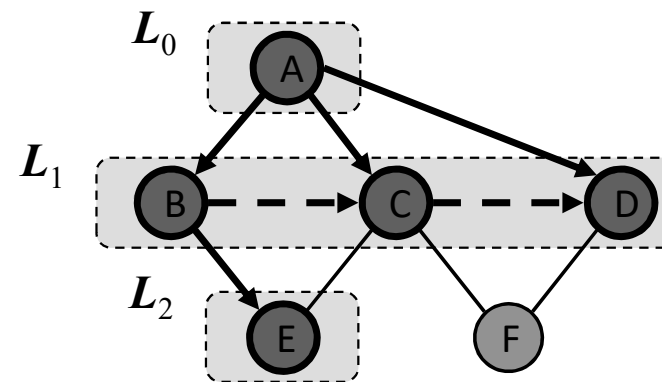
- We can specialize the DFS algorithm to find a simple cycle using the template method pattern
- We use a stack S to keep track of the path between the start vertex and the current vertex
- As soon as a back edge (v, w) is encountered, we return the cycle as the portion of the stack from the top to vertex w

```
Algorithm cycleDFS( $G, v, z$ )  
  setLabel( $v, VISITED$ )  
   $S.push(v)$   
  for all  $e \in G.incidentEdges(v)$   
    if getLabel( $e$ ) = UNEXPLORED  
       $w \leftarrow opposite(v, e)$   
       $S.push(e)$   
      if getLabel( $w$ ) = UNEXPLORED  
        setLabel( $e, DISCOVERY$ )  
        pathDFS( $G, w, z$ )  
         $S.pop(e)$   
      else  
         $T \leftarrow$  new empty stack  
        repeat  
           $o \leftarrow S.pop()$   
           $T.push(o)$   
        until  $o = w$   
        return  $T.elements()$   
   $S.pop(v)$ 
```

Depth-First Search – Follow in alphabet order



Breadth-First Search



Breadth-First Search (§ 12.3.3)

- Breadth-first search (BFS) is a general technique for traversing a graph
- A BFS traversal of a graph G
 - Visits all the vertices and edges of G
 - Determines whether G is connected
 - Computes the connected components of G
 - Computes a spanning forest of G
- BFS on a graph with n vertices and m edges takes $O(n + m)$ time
- BFS can be further extended to solve other graph problems
 - Find and report a path with the minimum number of edges between two given vertices
 - Find a simple cycle, if there is one

BFS Algorithm

- The algorithm uses a mechanism for setting and getting “labels” of vertices and edges

Algorithm *BFS*(*G*)

Input graph *G*

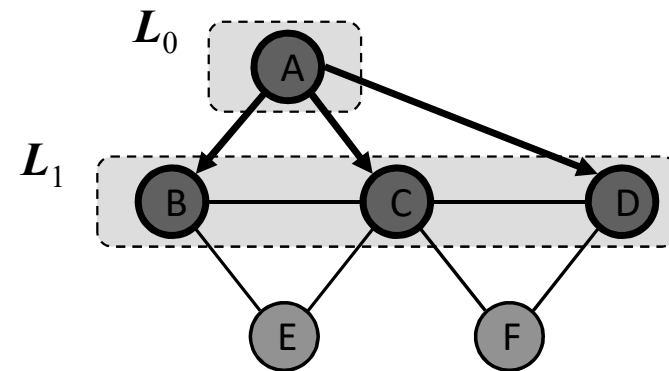
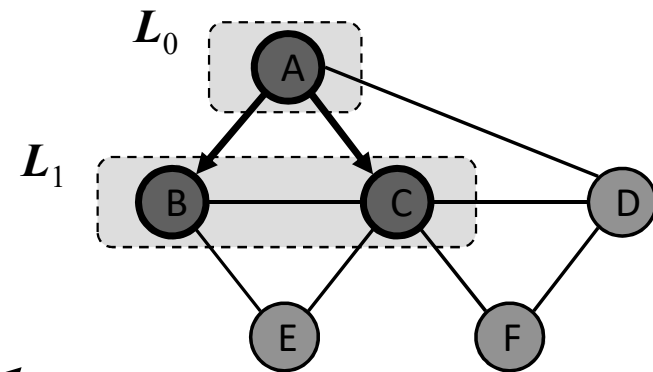
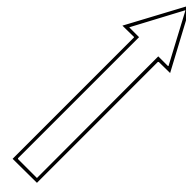
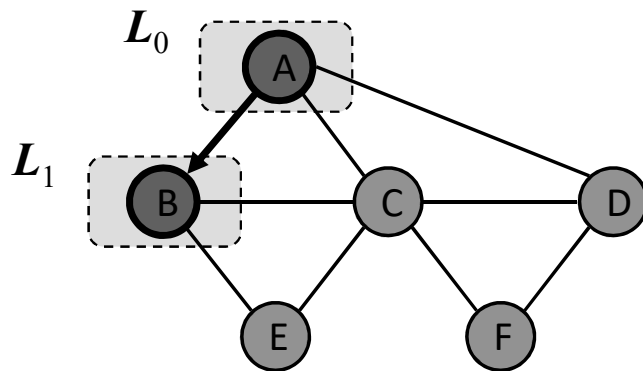
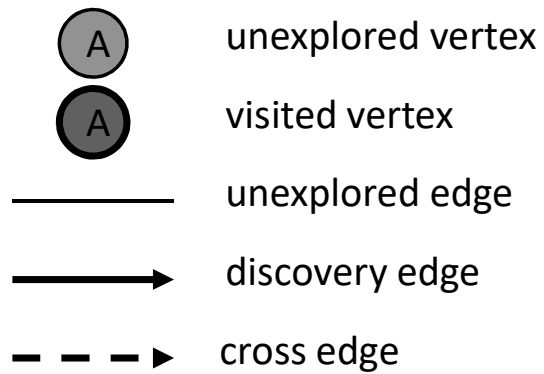
Output labeling of the edges and partition of the vertices of *G*

```
for all u ∈ G.vertices()
    setLabel(u, UNEXPLORED)
for all e ∈ G.edges()
    setLabel(e, UNEXPLORED)
for all v ∈ G.vertices()
    if getLabel(v) = UNEXPLORED
        BFS(G, v)
```

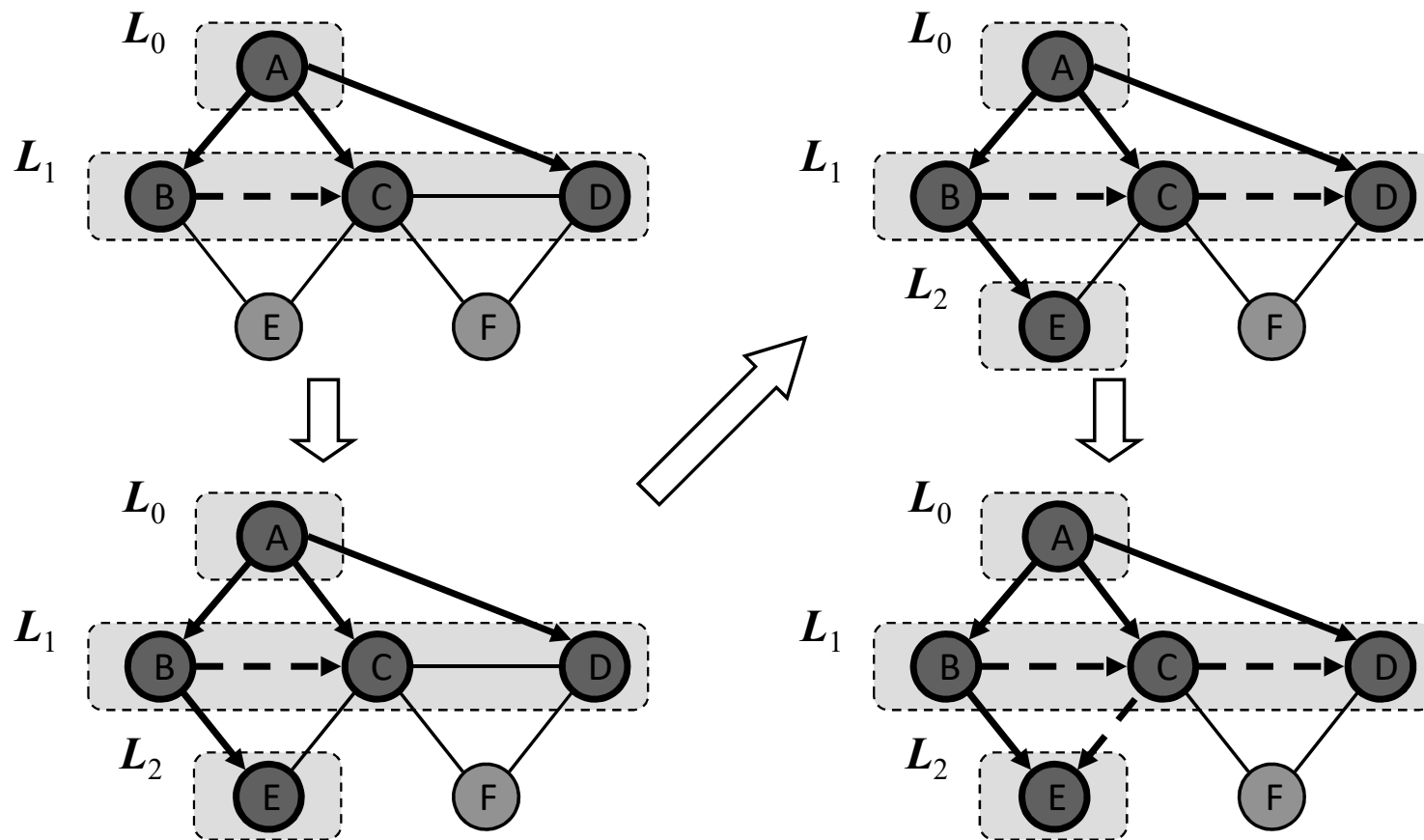
Algorithm *BFS*(*G*, *s*)

```
L0 ← new empty sequence
L0.insertLast(s)
setLabel(s, VISITED)
i ← 0
while ¬Li.isEmpty()
    Li+1 ← new empty sequence
    for all v ∈ Li.elements()
        for all e ∈ G.incidentEdges(v)
            if getLabel(e) = UNEXPLORED
                w ← opposite(v, e)
                if getLabel(w) = UNEXPLORED
                    setLabel(e, DISCOVERY)
                    setLabel(w, VISITED)
                    Li+1.insertLast(w)
                else
                    setLabel(e, CROSS)
    i ← i + 1
```

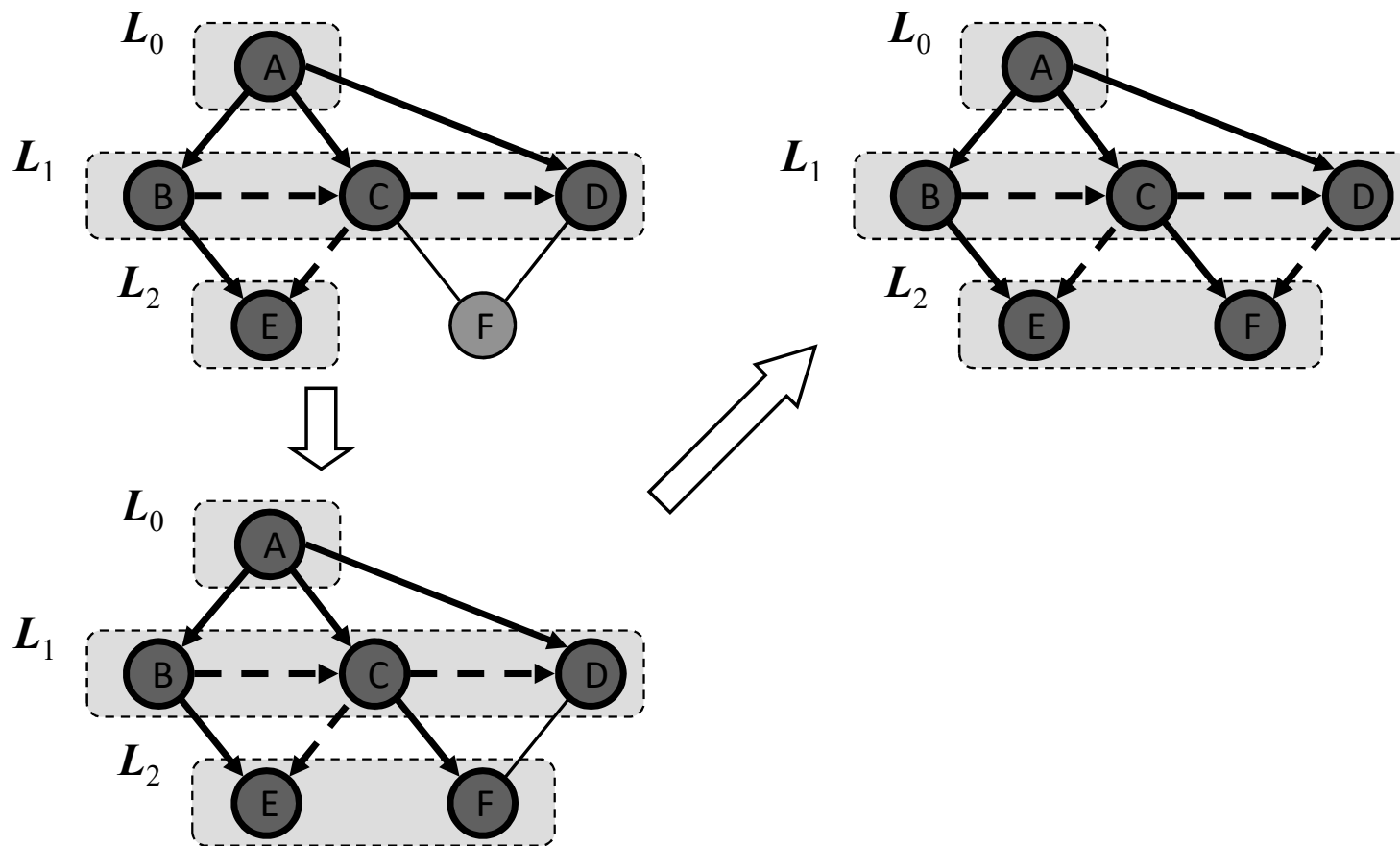

Example



Example (cont.)



Example (cont.)



Properties

Notation

G_s : connected component of s

Property 1

$BFS(G, s)$ visits all the vertices and edges of G_s

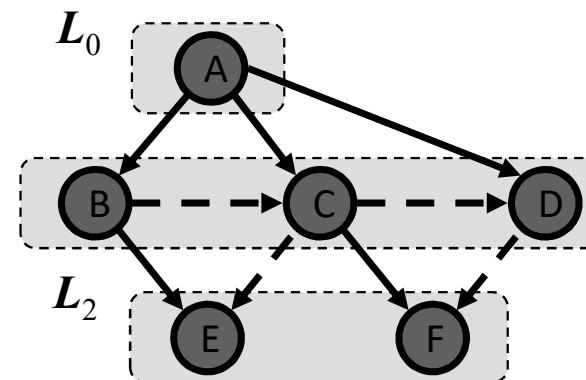
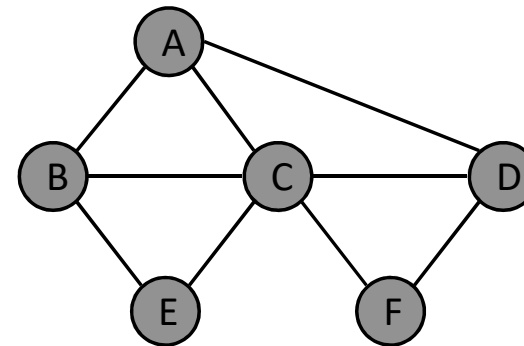
Property 2

The discovery edges labeled by $BFS(G, s)$ form a spanning tree T_s of G_s

Property 3

For each vertex v in L_i

- The path of T_s from s to v has i edges
- Every path from s to v in G_s has at least i edges



Analysis

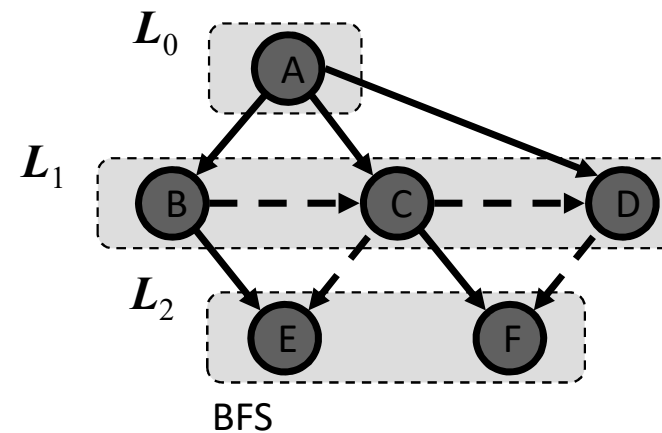
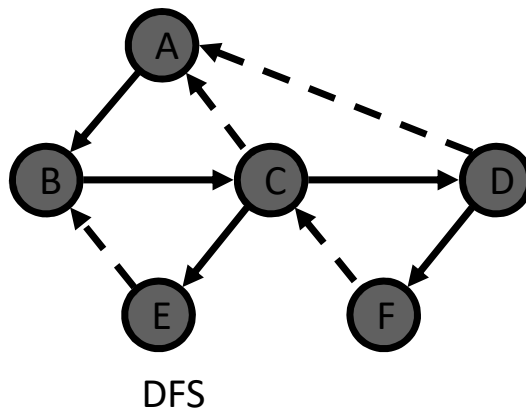
- Setting/getting a vertex/edge label takes $O(1)$ time
- Each vertex is labeled twice
 - once as UNEXPLORED
 - once as VISITED
- Each edge is labeled twice
 - once as UNEXPLORED
 - once as DISCOVERY or CROSS
- Each vertex is inserted once into a sequence L_i
- Method incidentEdges is called once for each vertex
- BFS runs in $O(n + m)$ time provided the graph is represented by the adjacency list structure
 - Recall that $\sum_v \deg(v) = 2m$

Applications

- Using the template method pattern, we can specialize the BFS traversal of a graph G to solve the following problems in $O(n + m)$ time
 - Compute the connected components of G
 - Compute a spanning forest of G
 - Find a simple cycle in G , or report that G is a forest
 - Given two vertices of G , find a path in G between them with the minimum number of edges, or report that no such path exists

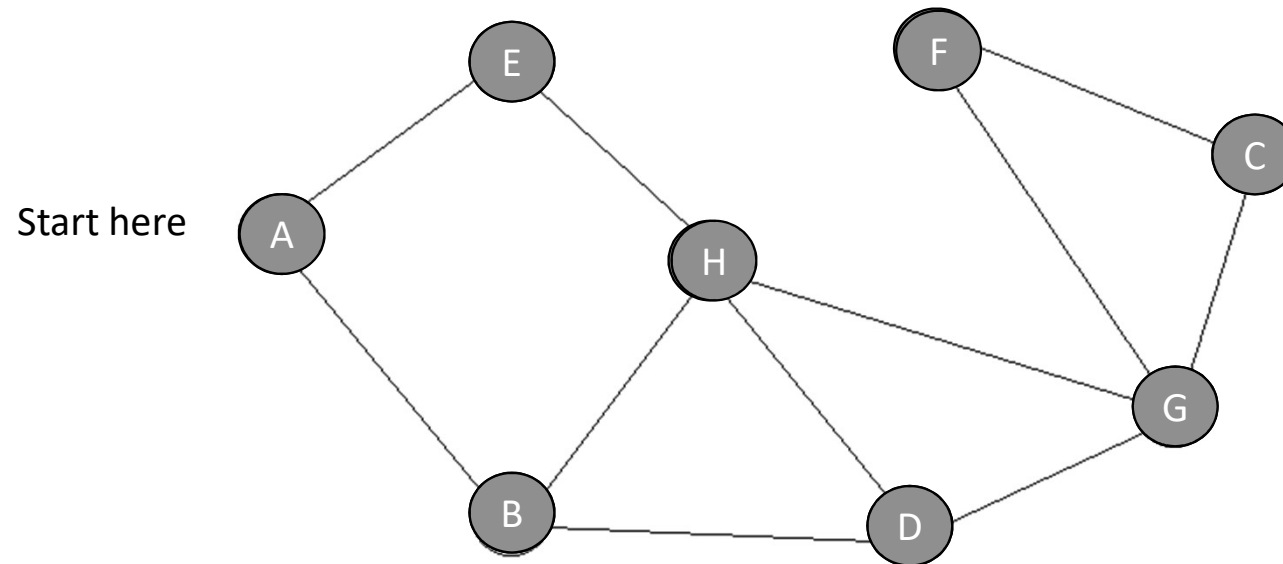
DFS vs. BFS

Applications	DFS	BFS
Spanning forest, connected components, paths, cycles	✓	✓
Shortest paths		✓



Breadth-First Search (in alphabet order)

Vertex of A is B & E



Queue: B
E
D
H
G
C
F

Result

Minimum Spanning Tree (MST)

- Connected and undirected graph
- Connects all the vertices together
- A single graph can have more than one minimum spanning tree
- Each edge will assign a weight

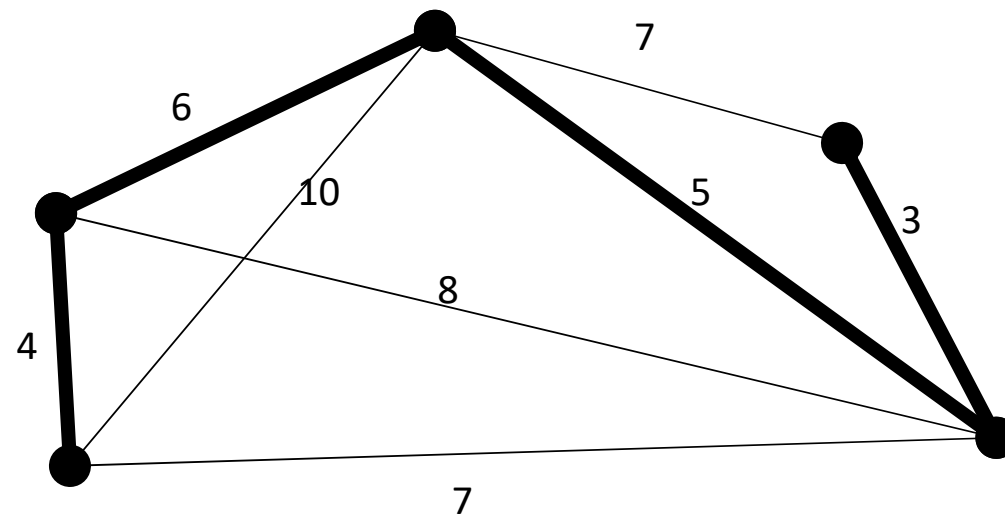
Shortest Path Algorithms

- The shortest path problem is about finding a path between 2 vertices in a graph such that the total sum of the edges weights is minimum.
- Minimal spanning tree algorithms are discussed in the following slides

Kruskal's algorithm

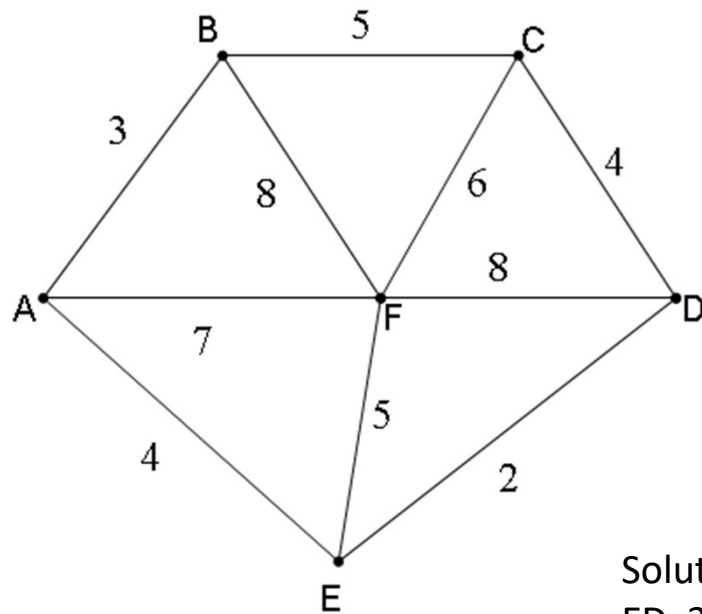
- Is a greedy algorithm in graph theory
- finds a minimum spanning tree for a connected weighted graph
 - Choose the minimum weight among all the edges and mark the chosen edge
 - Continue to choose the next minimum weight and mark the edge
 - Continue to choose and marking edge that does not create the circuit (NOTE: ensure the chosen edge does not forms a circuit)

Example



MST: $3+4+5+6=18$

Exercise



Solution

ED 2

AB 3

CD 4

AE 4

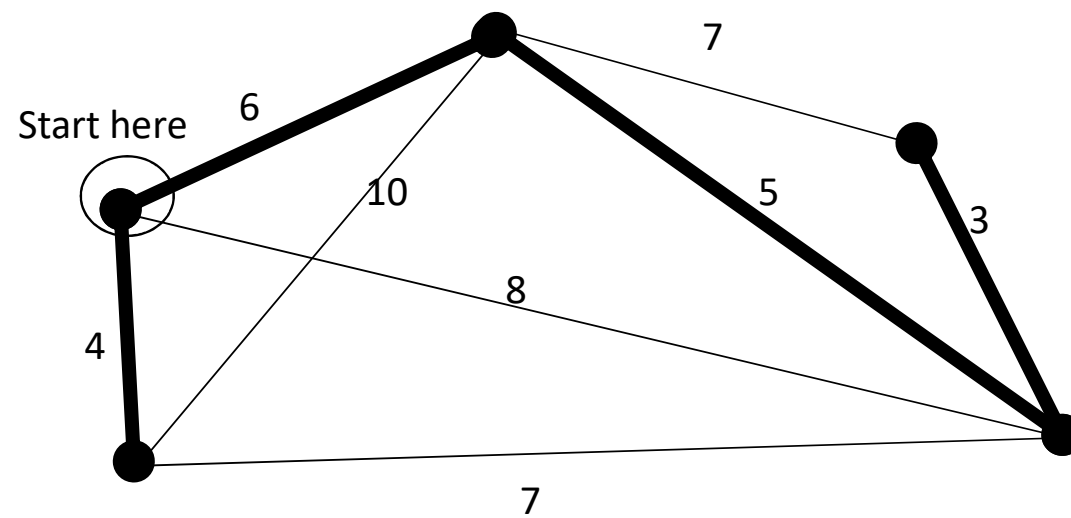
EF 5,

with Total weight of tree equal to 18

Prim's Algorithm

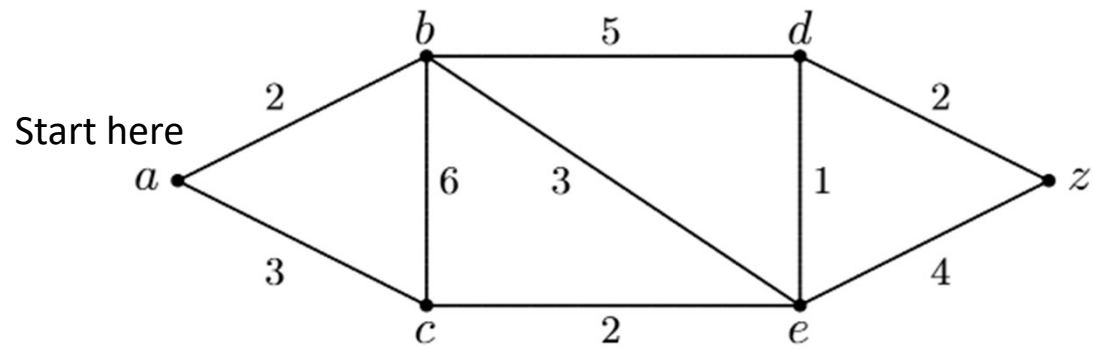
- Step 1: Choose any starting vertex. Look at all edges connecting to the vertex. Choose the one with the lowest weight
- Step 2: Look at all edges connected, choose the one with the lowest weight
- Step 3: Repeat step 2 until all vertices are in selected.

Example



MST: $4+6+5+3=18$

Exercise



Solution

ab 2

bc 3

ed 1

dz 2

ec 2

with Total weight of tree equal to 10

Dijkstra Algorithm

- Find shortest paths from a source vertex x to all other vertices
- is a solution to the single-source shortest path problem in graph theory.
- Works on both directed and undirected graphs.
- all edges must have nonnegative weights.

Algorithm

- Step 1: *Start with a vertex*
- Step 2: Find the lowest weighted path to another vertex
- Step 3: Repeat Step 2 until all vertices are selected

Example

- $A \rightarrow B \Rightarrow 4$
- $A \rightarrow C \Rightarrow 10$
- $A \rightarrow D \Rightarrow 10$
- $A \rightarrow E \Rightarrow 7$

